Foundations of Programming

Assignment No. : 11

PROBLEM STATEMENT:

Write a program in C to accept a number from the user and compute the following:

a. Square root of the number
b. Square of the number
c. Cube of the number
d. Check whether the number is prime
e. Factorial of the number
f. Prime factors of the number

OBJECTIVE:

• To understand the use of mathematical operations in C.

• To implement loops and conditional statements

. • To perform number-based computations.

• To enhance logical thinking using menu-driven programs.

THEORY:

Numbers and their mathematical properties play a vital role in programming. Operations such as square, cube, and square root are basic mathematical computations. Prime number checking involves verifying whether a number has exactly two divisors. Factorial is calculated by multiplying a sequence of integers. Prime factorization breaks a number into its prime components. In C programming, these operations can be efficiently implemented using loops, conditional statements, and mathematical functions.

PLATFORM: Linux

ALGORITHM:

1. Start
2. Input a number n from the user
3. Find square root:
   - Calculate square root of n
4. Find square:
   - square = n × n
5. Find cube:
   - cube = n × n × n
6. Check prime numb
   - Set count = 0
   - For i = 1 to n:
     - If n % i == 0 → count++
   - If count == 2 → number is Prime
   - Else → Not Prime
7. Find factorial:

- Set fact = 1
- For i = 1 to n:
    - fact = fact × i

8. Find prime factors:

- For i = 2 to n:
    - While n % i == 0:
        - Print i
        - n = n / i

9. Display all results
10. Stop

C PROGRAM:

```
#include <stdio.h>

#include <math.h>

int main() {

  int n, i, fact = 1, result;

  float squrt, square, cube;

  printf("Enter a number: ");

  scanf("%d", &n);

  if(n >= 0)

     squrt = pow(n, 0.5);

  else

     printf("Square root not defined for negative number\n");

 square = n * n;

 cube = n * n * n;

 if(n <= 1) {

     printf("Not a Prime Number\n");

  }

  else {

     int prime = 1;

     i = 2;

     while(i <= sqrt(n)) {

        if(n % i == 0) {

           prime = 0;

           break;

        }

        i++;
```

```
    }
    if(prime == 1)
        printf("Prime Number\n");
    else
        printf("Not a Prime Number\n");
}
if(n < 0) {
    printf("Factorial not defined for negative number\n");
}
else {
    fact = 1;
    for(i = 1; i <= n; i++)
        fact = fact * i;
}
if(n > 1) {
    int temp = n;
    printf("Prime Factors: ");
    i = 2;
    while(temp > 1) {
        if(temp % i == 0) {
            printf("%d ", i);
            temp = temp / i;
        }
        else
            i++;
    }
    printf("\n");
}
if(n >= 0)
 printf("Square Root = %.2f\n", squrt);
```

```
    printf("Square = %.2f\n", square);

    printf("Cube = %.2f\n", cube);

    if(n >= 0)

        printf("Factorial = %d\n", fact);

    return 0;

}
```

```c
#include <stdio.h>
#include <math.h>
int main() {
    int n, i, fact = 1, result;
    float squrt, square, cube;

    printf("Enter a number: ");
    scanf("%d", &n);
    if(n >= 0)
        squrt = pow(n, 0.5);
    else
        printf("Square root not defined for negative number\n");
   square = n * n;
   cube = n * n * n;
   if(n <= 1) {
        printf("Not a Prime Number\n");
    }
    else {
        int prime = 1;
        i = 2;

        while(i <= sqrt(n)) {
            if(n % i == 0) {
                prime = 0;
                break;
            }
            i++;
        }

        if(prime == 1)
            printf("Prime Number\n");
        else
            printf("Not a Prime Number\n");
    }
    if(n < 0) {
        printf("Factorial not defined for negative number\n");
    }
    else {
        fact = 1;
        for(i = 1; i <= n; i++)
            fact = fact * i;
    }
    if(n > 1) {
        int temp = n;
        printf("Prime Factors: ");
        i = 2;

        while(temp > 1) {
            if(temp % i == 0) {
                printf("%d ", i);
                temp = temp / i;
            }
            else
                i++;
        }
        printf("\n");
    }
    if(n >= 0)
        printf("Square Root = %.2f\n", squrt);

    printf("Square = %.2f\n", square);
    printf("Cube = %.2f\n", cube);

    if(n >= 0)
        printf("Factorial = %d\n", fact);
    return 0;
}
```

OUTPUT:

Enter a number: 25

Menu:

1. Square Root

2. Square

3. Cube

4. Check Prime

5. Factorial

6. Prime Factors

Enter your choice: 1

Square Root = 5.0000

Paste Screen Shot of Output

CONCLUSION:

Thus, the program to perform various mathematical operations on a number was successfully implemented using C programming. This program demonstrates the use of loops, conditional statements, switch-case, and mathematical functions.

FAQs:

1. Why is checking divisibility only up to n/2 sufficient in prime number verification, and how can this be optimized further?
2. Why is checking divisibility only up to n/2 sufficient in prime number verification, and how can this be optimized further is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
3. What are the limitations of using int data type for factorial calculation, and how can overflow be handled in C?
4. A data type specifies the kind of value a variable can store, such as integer, float, character, or string. Choosing the correct data type improves program accuracy, memory usage, and reliability.
5. How does the time complexity of prime factorization change with increasing input size?
6. How does the time complexity of prime factorization change with increasing input size is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
7. Why is the math.h library required for square root calculation, and what happens if it is not included?
8. Why is the math.h library required for square root calculation, and what happens if it is not included is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
9. How would the program behaviour change if a negative number is entered for factorial or square root operations?
10. How would the program behaviour change if a negative number is entered for factorial or square root operations is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
11. Explain the difference between checking a number for primality and generating prime factors of the same number.
12. Explain the difference between checking a number for primality and generating prime factors of the same number. is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.
13. How can recursion be used to calculate factorial instead of loops, and what are its advantages and disadvantages?
14. How can recursion be used to calculate factorial instead of loops, and what are its advantages and disadvantages is an important programming concept because it supports clear problem solving, structured logic, and correct implementation. Understanding it helps in writing programs that are easier to develop, test, and maintain.